

Injective PEPS Fundamental Theorem: Section 3 Route

2026-05-06

1 The source argument

The injective PEPS part of [MGRPG⁺18] proves the Fundamental Theorem for tensor networks on finite graphs without double edges and without self-loops. The proof is local. It does not slice a two-dimensional network into a long one-dimensional chain. Instead, it fixes an edge, blocks the graph around that edge into a three-site injective MPS, applies the one-dimensional injective theorem, and repeats this construction over all edges.¹

For a chosen edge $e = (u, v)$, all vertices other than u and v are grouped into the middle tensor. The two endpoint tensors and the middle tensor form a three-site MPS. Injectivity of the PEPS tensors implies injectivity of this blocked MPS. Since the two original PEPS represent the same state, the two blocked MPS represent the same state. The one-dimensional injective theorem therefore gives an algebra isomorphism between matrix insertions on the chosen bond. Equivalently, an arbitrary matrix X inserted on that bond in one network is matched by a matrix on the corresponding bond in the other network.

After choosing the resulting edge gauges for every edge and absorbing them into one tensor family, the paper obtains the following stronger comparison: for every edge and every matrix X , inserting X on that edge in the first PEPS gives the same state as inserting X on the same edge in the modified second PEPS.²

2 Current formal representation

The finite graph, edge, incident-edge, tensor, virtual configuration, state coefficient, equality of states, and vertex injectivity are represented in `TNLean/PEPS/Defs.lean`. Vertex injectivity is the linear independence of the local physical vectors indexed by local virtual configurations, matching the paper's statement

¹The edge-blocking construction is `Papers/1804.04964/paper_normal.tex:981--1009`. The equality of the two PEPS before blocking is at `Papers/1804.04964/paper_normal.tex:1010--1036`. The application of the three-site injective MPS theorem is at `Papers/1804.04964/paper_normal.tex:1037--1065`.

²This is the displayed equation labelled `eq:inj_equal_edge` in `Papers/1804.04964/paper_normal.tex:1037--1065`.

that each tensor is injective as a map from the virtual space to the physical space.

The local linear-algebraic realization of virtual operations at an injective vertex is represented in `TNLean/PEPS/VirtualInsertion.lean`. The chosen left inverse of the local tensor map and the resulting physical realization of local virtual endomorphisms correspond to the same observation used in the three-site MPS proof: a virtual operation can be represented by a physical operation on a neighboring site. The basis computation `TNLean.PEPS.localIncidentMatrixOp.single` and its tensor-map form `TNLean.PEPS.localTensorMap.localIncidentMatrixOp.single` identify the one-edge matrix action on a single local virtual boundary with the expected sum over the new distinguished edge index.

The edge-centered blocking data are represented in `TNLean/PEPS/Blocking.lean`. The existing declarations split local virtual configurations at a distinguished incident edge and split the vertex product into the left endpoint, the middle region, and the right endpoint. The edge-blocked coefficient is proved equal to the original PEPS coefficient.

The next formal layer is the matrix-insertion comparison. The declarations `TNLean.PEPS.EdgeInsertedBoundaryConfig`, `TNLean.PEPS.edgeBoundaryToInsertedBoundaryConfig`, `TNLean.PEPS.EdgeDiagonalInsertedBoundaryConfig`, `TNLean.PEPS.edgeBoundaryConfigEquivDiagonalInsertedBoundaryConfig`, `TNLean.PEPS.edgeMiddleConfigToOpenMiddleConfig`, `TNLean.PEPS.edgeOpenMiddleConfigToMiddleConfig`, `TNLean.PEPS.edgeMiddleConfigEquivOpenMiddleConfig`, `TNLean.PEPS.edgeOpenMiddleWeight`, `TNLean.PEPS.edgeBoundaryLeftIndexEquivInsertedBoundaryConfig`, `TNLean.PEPS.edgeBoundaryRightIndexEquivInsertedBoundaryConfig` and `TNLean.PEPS.edgeInsertedCoeff` represent the open edge contraction used when an arbitrary matrix is inserted on the distinguished edge. These definitions are the formal counterpart of the matrix insertion appearing before and after the equation labelled `eq:inj_equal_edge`. The two maps `TNLean.PEPS.edgeMiddleConfigToOpenMiddleConfig` and `TNLean.PEPS.edgeOpenMiddleConfigToMiddleConfig` have been packaged as the finite equivalence `TNLean.PEPS.edgeMiddleConfigEquivOpenMiddleConfig`. This isolates the reindexing needed for the identity-insertion specialization. The current Lean layer includes the corresponding middle-tensor reindexing theorem `TNLean.PEPS.edgeMiddleWeight.eq.edgeOpenMiddleWeight`. The diagonal summand has also been isolated in `TNLean.PEPS.edgeInsertedCoeff.identity.diagonal.summand`, and `TNLean.PEPS.edgeInsertedCoeff.identity.offDiagonal.summand` records the pointwise off-diagonal vanishing. The full identity-insertion specialization is `TNLean.PEPS.edgeInsertedCoeff.identity`; its proof uses `TNLean.PEPS.edgeBoundaryConfigEquivDiagonalInsertedBoundaryConfig` to reindex the finite inserted-boundary sum along the diagonal.

3 Remaining mathematical obligations

Systematic positive-bond restriction. Section 3 uses the standing assumption that injective PEPS have nonzero-dimensional virtual bond spaces. If this assumption is omitted, a zero-dimensional edge empties a virtual configuration, makes an injectivity or equality hypothesis vacuous, and leaves

the conclusion as a genuine constraint that can fail. The affected statements are the edge-blocked three-site injectivity (#1366), the physical-to-virtual recovery (#1370), the edge-blocked insertion-algebra isomorphism (#1367), the global gauge consistency, the gauge uniqueness statement, and the headline theorem `TNLean.PEPS.fundamentalTheoremPEPS`. The restored hypothesis is $\forall e, 0 < \dim(e)$, for both tensors where both occur. The two-block comparison `TNLean.PEPS.two injective tensor insertion comparison` and the post-absorption equality are *not* affected: there the conclusion degenerates with the hypotheses, so the statement is vacuously true. The headline counterexample is `TNLean.PEPS.FundamentalTheoremCounterexample.fundamentalTheoremPEPSStatement false`.

The following statements are still needed before the injective PEPS Fundamental Theorem can be proved in the manner of the paper.³

1. Vertex injectivity must be shown to pass to the edge-blocked three-site object. This is the formal version of the paper's assertion that the regrouped MPS is injective. The middle physical index for this regrouped object is now explicit as `TNLean.PEPS.EdgeMiddlePhysicalConfig`, and the middle contractions have the middle-indexed forms `TNLean.PEPS.edgeMiddleWeightOn` and `TNLean.PEPS.edgeOpenMiddleWeightOn`. In the blueprint the injectivity statement is `TNLean.PEPS.EdgeBlockedThreeSiteInjective`. The singleton base case is now formalized by `TNLean.PEPS.singletonRegionTensorFamily` and `TNLean.PEPS.IsVertexInjective.singletonRegionTensorInjective`: for a one-vertex block $\{v\}$, $T_{A,\{v\}}(\eta, \sigma) = A_v(\eta, \sigma)$, hence vertex injectivity gives $\text{inj}(T_{A,\{v\}})$. The bridge to the abstract region-injectivity predicate is recorded by `TNLean.PEPS.SingletonRegionInjectivityComparison` and `TNLean.PEPS.SingletonRegionInjectivityFromVertexInjectivity.ofSingletonComparison`. With the abstract union-closure lemma, this gives every nonempty finite region by `TNLean.PEPS.RegionInjectivityUnionClosure.finset injective of singletons`: $R = \bigcup_{v \in R} \{v\}$ and $\kappa(\{v\})$ for every $v \in R$ imply $\kappa(R)$. The corresponding nonempty-middle edge-middle tensor consequence is recorded in `thm:peps_edgeMiddleTensorInjective_of_singletons`: for $M_e = V \setminus \{u, v\}$, the assumptions $M_e \neq \emptyset$ and $M_e = \bigcup_{w \in M_e} \{w\}$ give $\kappa(M_e)$; the edge-middle comparison then gives injectivity of the middle tensor. The endpoint reduction then gives the one-edge three-site statement `thm:peps_edgeBlockedThreeSiteInjective_of_singletons`, and applying this to all edges satisfying $M_e \neq \emptyset$ gives `thm:peps_edgeBlockedThreeSiteInjective_all_of_singletons`. The graph-cardinality variant `TNLean.PEPS.EdgeMiddleRegionInjectivityComparison.edgeBlockedThreeSiteInjective all two lt card` packages the common case $2 < |V|$: since $|M_e| = |V| - 2 > 0$ for every edge, the nonempty-middle hypothesis is automatic. Vertex injectivity supplies the two endpoint assertions by `TNLean.PEPS.IsVertexInjective.edgeBlockedEndpointTensorMaps injective` and, for all edges at once, by `TNLean.PEPS.IsVertexInjective.edgeBlockedEndpointTensorMaps injective all`, and `TNLean.PEPS.IsVertexInjective.edgeBlockedThreeSiteInjective of middle` reduces the remaining work to the middle-block injectivity statement. Its

³This list and the ledger below are the status record referenced by the Section 3 docstrings and blueprint comments.

all-edge form is `TNLean.PEPS.IsVertexInjective.edgeBlockedThreeSiteInjective`all`of`middle``.

The comparison `TNLean.PEPS.EdgeMiddleRegionInjectivityComparison` records the conditional passage from finite-region injectivity of $V \setminus \{u, v\}$ to the edge-middle tensor family. Its theorem `TNLean.PEPS.EdgeMiddleRegionInjectivityComparison.edgeBlockedThreeSiteInjective`all`of`middle``

then combines this comparison with vertex injectivity to obtain the edge-blocked three-site assertion at one edge. The all-edge middle-tensor comparison is `TNLean.PEPS.EdgeMiddleRegionInjectivityComparison.edgeMiddleTensorInjective`all``. The all-edge conditional form is `TNLean.PEPS.EdgeMiddleRegionInjectivityComparison.edgeBlockedThreeSiteInjective`all`of`middle``.

The finite induction on exposed middle vertices is separated as `TNLean.PEPS.FiniteRegionKernelDescent`: for kernel conditions $K_c(S)$ attached to a coefficient family $c = (c_\rho)_\rho$, the implications $K_c(S) \Rightarrow K_c(S \setminus \{j\})$ imply $K_c(M_e) \Rightarrow K_c(\emptyset)$.

The edge-middle instance of this coefficient argument is recorded by `TNLean.PEPS.EdgeMiddleKernelDescentData`: the zero relation $\sum_\rho c_\rho T_{A, M_e}^\rho(\tau) = 0$ for every middle physical index τ gives $K_c(M_e)$, and $K_c(\emptyset)$ gives $c_\rho = 0$ for every ρ . Consequently `TNLean.PEPS.EdgeMiddleKernelDescentData.edgeMiddleTensorInjective`all``

proves `inj(TA, Me)` from such a datum, and `TNLean.PEPS.IsVertexInjective.edgeBlockedThreeSiteInjective`all`of`middle`` combines it with the endpoint injectivity assertions. The implication from vertex injectivity to middle-block injectivity is false without a positivity hypothesis on the virtual bond dimensions. If some complement bond makes `TNLean.PEPS.EdgeComplementConfig` empty, then the middle tensor family is identically zero (`TNLean.PEPS.edgeMiddleTensorFamily`eq`zero`of`isEmpty``), and the family is not linearly independent when the boundary-label index is nonempty (`TNLean.PEPS.not`edgeMiddleTensorInjective`of`isEmpty``).

A four-cycle with a zero-dimensional middle-to-middle edge gives the corresponding mathematical example in which vertex injectivity can survive vacuously at vertices incident to that bond. The source avoids this because injective PEPS have nonzero-dimensional virtual bond spaces. The hypothesis $\forall f, 0 < \dim(f)$ is therefore part of `TNLean.PEPS.IsVertexInjective.edgeBlockedThreeSiteInjective`; the two lemmas above record the formal non-injectivity obstruction.

The mathematical step represented by the restored theorem is the contraction theorem used in the paper: contracting vertex-injective tensors over that finite middle region gives an injective blocked tensor. The source proves this in one step from the one-sided inverse, where the inverse of a contraction of injective tensors is the contraction of the inverses up to the connecting bond dimension (`Papers/1804.04964/paper_normal.tex`, lines 205–250). The kernel descent recorded above is the finite-induction route towards that one-step fact, not a separate argument in the source. The full transfer is stated as `TNLean.PEPS.IsVertexInjective.edgeBlockedThreeSiteInjective` with the positive-bond hypothesis; its proof consists of the three facts below.

The per-vertex one-sided-inverse step is expressed by `TNLean.PEPS.IsVertexInjective.localCoeff`eq`zero`of`contract`to`zero``: a coefficient family on the local virtual configurations at a vertex v that contracts to the zero physical vector vanishes. The middle tensor is

a genuine per-vertex product, $T_{A,M_e}(\rho, \tau) = \sum_{\zeta} \prod_{v \in M_e} A_v(\zeta, \tau_v)$, so a single vertex j is removed by $\prod_{v \in M_e} = A_j \cdot \prod_{v \in M_e \setminus \{j\}}$. Three facts instantiate the kernel descent from vertex injectivity:

- (a) the star-bond split equivalence `TNLean.PEPS.edgeComplementConfigSplitAt`, decomposing the complement bond configuration into the star bonds of j (its local virtual configuration) and the remaining bonds, with a first-component law matching `edgeComplementValue`; the shared bond between two middle vertices is read back from the star factor, which is why the peeled coefficient genuinely depends on j 's local virtual configuration;
- (b) the partial-contraction kernel condition over a finite open region S together with its one-vertex deletion $K_c(S) \Rightarrow K_c(S \setminus \{j\})$, obtained by fixing the exposed bonds, ranging the open physical leg of j , and applying `TNLean.PEPS.isVertexInjective.localCoeff`eq`zero`of`contract`zero`;
- (c) the terminal isolation $K_c(\emptyset) \Rightarrow c = 0$ over the residual endpoint boundary labels, by surjectivity of the boundary labelling, which is where the positive bond dimensions are used.

These assemble into `TNLean.PEPS.edgeMiddleKernelDescentData` and close the target by the existing reductions.

2. The two internal steps in Lemma `inj_isomorph` must be formalized for the edge-blocked three-site MPS. The local endpoint part of the virtual-to-physical realization $X \mapsto O_1, O_2$ is formalized by `thm:peps_virtualInsertionPhysicalRealization`: vertex injectivity turns the one-leg virtual matrix action at either endpoint into a physical operator on that endpoint. The endpoint coefficient-level direction is formalized by `thm:peps_edgeInsertedCoeff_endpointPhysicalRealization`: after fixing ordinary edge-boundary data, the left and right physical operators supply the independent inserted-edge index at the corresponding endpoint. On the left endpoint the formal statement uses the transpose of the inserted matrix, because the ordinary boundary supplies the right bond index. The source recovery $O_1, O_2 \mapsto W$ is stated as `TNLean.PEPS.physical`to`virtual`insertion`; its proof remains open and is tracked by issue #1370. As for the middle-block injectivity, this statement is false without a positive-bond hypothesis: a zero-dimensional edge incident to an endpoint empties the edge boundary configuration, so the resonate hypothesis holds vacuously while the recovery conclusion stays a genuine constraint that fails for a nontrivial right-endpoint operator. The formal counterexample is `TNLean.PEPS.PhysicalToVirtualCounterexample.physical`to`virtual`insertion`statement`false`, and the hypothesis  $\forall f, 0 < \dim(f)$  has been restored on `TNLean.PEPS.physical`to`virtual`insertion`. This is a local correction: the source recovery lemma is stated for injective three-site tensor networks with a vector space assigned to the edge `Papers/1804.04964/paper_normal.tex:250,254--255`,

while the formal statement also admits the degenerate case $\dim(f) = 0$. Requiring $0 < \dim(f)$ makes the source's nonzero virtual edge space explicit. The remaining proof needs, in addition, the local realization engine to be parametrized by endpoint injectivity (available from `TNLean.PEPS.EdgeBlockedThreeSiteInjective`) rather than global vertex injectivity, and a one-sided inverse for the right-endpoint-plus-middle contraction that extracts the recovered matrix. Its local endpoint recovery component is now formalized by `TNLean.PEPS.edgeEndpointLocalVirtualOpOfPhysicalOp` of `projected`realization`eq`: once the compressed endpoint actions are known to realize the same bond matrix, the local virtual pullbacks recover that matrix on the shared bond. The image-preserving conditional form `TNLean.PEPS.edgePhysicalToVirtualInsertion` of `projected`realization`eq` has the endpoint conclusion of the source theorem: if those same projected realizations hold and the two endpoint physical operators preserve the local tensor images, then $O_1\Phi_u = \Phi_u T_{u,e}(M^\top)$ and $O_2\Phi_v = \Phi_v T_{v,e}(M)$. This conditional endpoint theorem is not a replacement for `TNLean.PEPS.physical`to`virtual`insertion`: the remaining paper step is to obtain the common matrix, the two projected-realization identities, and the image-preservation hypotheses from equality of the neighboring physical actions on the edge-blocked three-site state.

3. Equality of PEPS states must be upgraded to equality of edge-inserted coefficients after the three-site reduction. This is the step that produces the matrix-algebra isomorphism on the chosen bond. The formally stated theorem is `TNLean.PEPS.isEdgeBlockedInsertionAlgebraIsomorphism`; its proof remains open and is tracked by issue #1367.
4. The algebra isomorphism must be converted into an invertible edge gauge, using the same finite-dimensional matrix-algebra argument as the one-dimensional injective theorem. In the blueprint this is the not-yet-formalized theorem `thm:peps_edgeGaugeFromInsertionAlgebraIsomorphism`.
5. After all edge gauges have been absorbed into the second tensor family, producing the modified tensors $\tilde{B}_v$, the formalization must prove the analog of `eq:inj_equal_edge`: arbitrary insertions of the same matrix on the same edge give equal PEPS states for the first tensor family and the modified second tensor family. In the blueprint the absorption step is the not-yet-formalized theorem `thm:peps_edgeGaugeAbsorption`, and the post-absorption insertion equality target is now named formally by `TNLean.PEPS.PostAbsorptionEdgeInsertionEquality`. Constructing that predicate from the absorbed edge gauges remains the not-yet-formalized theorem `thm:peps_postAbsorptionEdgeInsertionEquality`.
6. The generalized two-injective-tensor comparison from `lem:inj_equal_tensors_2` must be formalized. It says that equality of all virtual-bond insertions for two pairs of injective tensors forces $A_1 = \lambda B_1$ and $A_2 = \lambda^{-1} B_2$.

In the blueprint this is split into the scalar-reduction substep `thm:peps_twoInjectiveGaugeScalarReduction` and the formally stated comparison theorem `TNLean.PEPS.two`injective`tensor`insertion`comparison`. The formal statement is an abstract version with nonempty spectator external boundary spaces; the source lemma is recovered by taking these spaces to be one-point spaces. The rank-one cancellation after coefficientwise product equality is now formalized by `TNLean.PEPS.twoBlockReciprocalScalarProportional`of`pointwise`mul`eq`: if $A_1(\eta_1, \mu, \sigma_1)A_2(\eta_2, \nu, \sigma_2) = B_1(\eta_1, \mu, \sigma_1)B_2(\eta_2, \nu, \sigma_2)$ for all indices, then injectivity of A_1 and A_2 gives the reciprocal scalar proportionality. The one-shared-bond coefficient-separation case is now formalized by `TNLean.PEPS.two`injective`tensor`insertion`comparison`singletonBond`: inserting the matrix unit $E_{\mu,\nu}$ extracts the single product $A_1(\eta_1, \mu, \sigma_1)A_2(\eta_2, \nu, \sigma_2)$, and the pointwise product cancellation theorem then gives reciprocal scalar proportionality. The coefficientwise product separation above is on the source path only in the single-shared-bond case (`TNLean.PEPS.two`injective`tensor`insertion`comparison`singletonBond`), where a matrix-unit insertion extracts the product directly. In the many-bond case the source does not separate the product: it inverts  $A_2$ , reads off the gauges  $Z, U, W$  on three leg pairs, and uses their identity-form compatibility to force each gauge to be a scalar (`TNLean.PEPS.threeLeg`residual`forms`scalar`, already proved; [Papers/1804.04964/paper_normal.tex](#), lines 1157–1204). The remaining proof of `TNLean.PEPS.two`injective`tensor`insertion`comparison` is the inversion and leg-regrouping that produce Z, U, W from the one-bond insertions. This is tracked by issue #1361.

7. The final two-tensor comparison must then be applied by blocking one vertex against its complement. The paper uses this to prove $A_i = \lambda_i \hat{B}_i$ at each vertex, and then incorporates the scalars into the local gauges. This specialization is now represented by `TNLean.PEPS.one`vertex`complement`comparison`. The formal statement uses the abstract two-block notation with nonempty external boundary spaces; the source comparison is obtained from the one-vertex and complement blocks after `eq:inj_equal_edge`.
8. The boundary-insertion argument must remove the separate bond-dimension-identification hypothesis in `TNLean.PEPS.fundamentalTheorem`PEPS`of`bondDim`. This is the remaining difference between that conditional theorem and the source theorem; it is tracked by issue #874.

4 Source-to-formalization ledger

The following ledger records the present status of the Section 3 proof route. It is meant to prevent the proof from drifting away from the source argument.

- `eq:block_to_mps` edge blocking: `edgeBlockedCoeff_eq_stateCoeff` is locally proved; the injectivity bridge is stated as the positive-bond theorem `TNLean.PEPS.IsVertexInjective.edgeBlockedThreeSiteInjective`. The middle physical index of this three-site object is formalized by `TNLean.PEPS.EdgeMiddlePhysicalConfig`. The named injectivity statement is `TNLean.PEPS.EdgeBlockedThreeSiteInjective`, and the endpoint assertions are proved from vertex injectivity, both for a single chosen edge and uniformly over all edges. The comparison from edge-middle region injectivity to edge-middle tensor injectivity is also available uniformly over all edges. The all-edge conditional bridge from middle-region injectivity is `TNLean.PEPS.EdgeMiddleRegionInjectivityComparison.edgeBlockedThreeSiteInjective.all`.
- Identity insertion on the chosen bond: `thm:peps_edgeInsertedCoeff_identity` is formalized by finite diagonal reindexing.
- Lemma `inj_isomorph, X  $\mapsto$  O1, O2`: `thm:peps_virtualInsertionPhysicalRealization` formalizes the endpoint realization, and `thm:peps_edgeInsertedCoeff_endpointPhysicalRealization` formalizes the endpoint coefficient expansion. These are tracked by issues #1369 and #1995.
- Lemma `inj_isomorph, O1, O2  $\mapsto$  W`: `TNLean.PEPS.physical'to'virtual'insertion` states the source recovery step; its proof remains open and is tracked by issue #1370. The endpoint projected-recovery component for a common bond matrix is formalized by `TNLean.PEPS.edgeEndpointLocalVirtualOpOfPhysicalOp'eq'of'projected'realization`. The conditional theorem `TNLean.PEPS.edgePhysicalToVirtualInsertion'of'projected'realization'eq` additionally removes the endpoint projectors under explicit image-preservation hypotheses. The missing paper step is to obtain these hypotheses, and the common bond matrix, from equality of the two neighboring physical actions on the edge-blocked state. Thus the conditional endpoint theorem is a component of issue #1370, not the closure of that source-level obligation.
- Matrix-insertion algebra isomorphism: `TNLean.PEPS.isEdgeBlockedInsertionAlgebraIsomorphism` states the edge-blocked matrix-algebra correspondence and its inserted-coefficient equality. Its proof is tracked by issue #1367.
- Edge gauge from the algebra isomorphism: `thm:peps_edgeGaugeFromInsertionAlgebraIsomorphism`, tracked by issue #1368.
- Absorption of edge gauges into B : `thm:peps_edgeGaugeAbsorption`, tracked by issue #1363.
- Equation `eq:inj_equal_edge`: `thm:peps_postAbsorptionEdgeInsertionEquality`, tracked by issue #1364.
- Lemma `inj_equal_tensors_2`, scalar step: `thm:peps_twoInjectiveGaugeScalarReduction`, tracked by issue #1362.

- Lemma `inj_equal_tensors_2`, comparison: `TNLean.PEPS.two`injective`tensor`insertion`comparison` states the abstract two-block insertion comparison with nonempty external boundary spaces and reciprocal scalar proportionality. The source lemma is the one-point external-boundary specialization; the proof is tracked by issue #1361.
- One vertex versus its complement: `TNLean.PEPS.one`vertex`complement`comparison` records the application of the generalized two-block comparison to a selected vertex and its complement, under the abstract nonempty external-boundary formulation. The source proof is tracked by issue #1360.
- Final gauge equivalence and uniqueness: `thm:fundamentalTheorem_PEPS_of_bondDim` and the uniqueness theorem, tracked by issues #780, #842, and the boundary-insertion issue #874.

5 Blueprint diagrams

The blueprint now uses semantic tensor-network commands for the Section 3 moves that should remain visually attached to the proof.

- `\TNPEPSEdgeBlockingReduction` depicts the chosen edge and the passage to the three-site chain. Its endpoint labels follow `eq:block_to_mps`: the two circled endpoints are A'_1 and A'_2 , while the boxed complement is A'_3 .
- `\TNPEPSEdgeInsertedCoeff` depicts the single edge-blocked contraction with one virtual matrix X inserted on the chosen bond. This is the diagram attached to the definition of the inserted-edge coefficient, before comparing two tensor networks.
- `\TNPEPSThreeSiteInsertionComparison` depicts the comparison of a virtual insertion in the two three-site chains, as in the statement of `lem:inj_isomorph`.
- `\TNPEPSInsertionPhysicalRealization` depicts the realization of a virtual insertion as a physical operation on either neighboring site.
- `\TNPEPSPhysicalToVirtualInsertion` depicts the converse step in which two neighboring physical operations determine a virtual operation on the shared bond.
- `\TNPEPSEdgeInsertionEquality` depicts the full PEPS comparison after the edge gauges have been absorbed into the second tensor family, matching the five-vertex graph and the distinguished $(2, 5)$ -edge insertion in `eq:inj_equal_edge`.
- `\TNPEPSEdgeGaugeAbsorption` depicts the local absorption of incident oriented edge gauges into a B -tensor, producing $\tilde{B}_v$.

- `\TNPEPSTwoInjectiveTensorInsertionComparison` depicts the generalized two-injective-tensor comparison with an arbitrary matrix inserted on a shared virtual bond.
- `\TNPEPSTwoInjectiveGaugeScalarReduction` depicts the residual virtual operators Z, U, W that appear after applying an inverse for one injective block in the proof of `lem:inj_equal_tensors_2`.
- `\TNPEPSOneVertexComplementComparison` depicts the final one-vertex-versus-complement comparison used to prove $A_v \propto \tilde{B}_v$. It follows the source paper’s displayed comparison by drawing the one-vertex enlargement and the smaller injective region as tensor-network tensors, rather than as an auxiliary formal block.

These diagrams deliberately keep the local dot convention used in the blueprint, but they preserve the distinctions that matter in the paper: tensor sites, blocked regions, virtual matrix insertions, and physical operations are not drawn as the same object.

6 Conclusion

The current formalization has reached the edge-blocked coefficient and has begun the matrix-insertion layer needed for the Section 3 proof. It has not yet proved the three-site injectivity transfer, the insertion comparison, the resulting edge gauge construction, or the final one-vertex-versus-complement comparison. Those are the remaining mathematical steps required to turn the current PEPS definitions into a proof of the injective PEPS Fundamental Theorem faithful to [MGRPG⁺18, Section 3].

References

- [MGRPG⁺18] András Molnár, José Garre-Rubio, David Pérez-García, Norbert Schuch, and J. Ignacio Cirac. Normal projected entangled pair states generating the same state. *New Journal of Physics*, 20(11):113017, 2018.